Sortieralgorithmen - Übungsaufgaben und Lösungen

1. Bubble Sort

Aufgabe 1: Ablauf analysieren
Gegeben ist die Liste: [5, 2, 9, 1, 5, 6]. Wende den Bubble Sort Algorithmus schrittweise an und notiere die Liste nach jedem Durchgang.

Aufgabe 2: Implementierung
Schreibe eine Funktion in einer Programmiersprache deiner Wahl, die Bubble Sort implementiert.

****Lösung zu Aufgabe 1:****
1. [2, 5, 1, 5, 6, 9]
2. [2, 1, 5, 5, 6, 9]
3. [1, 2, 5, 5, 6, 9] (Sortiert)

****Lösung zu Aufgabe 2 (Python):****
```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
```

---

2. QuickSort

Aufgabe 1: Pivot-Element-Strategie
Erkläre, wie sich die Wahl des Pivot-Elements auf die Laufzeit von QuickSort auswirkt.

Aufgabe 2: Implementierung
Schreibe eine QuickSort-Funktion.

****Lösung zu Aufgabe 1:****

- Ein schlechtes Pivot (z. B. kleinstes oder größtes Element) kann zu einer Laufzeit von O(n^2) führen.
- Ein zufälliges oder medianbasiertes Pivot sorgt für bessere Leistung mit O(n log n).

****Lösung zu Aufgabe 2 (Python):****

```
def quicksort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quicksort(left) + middle + quicksort(right)
```

---

3. Merge Sort

Aufgabe 1: Zerlegung nachvollziehen

Zeichne die Zerlegung des Arrays [8, 4, 7, 3, 2, 6, 5, 1] bei der Anwendung von Merge Sort.

Aufgabe 2: Implementierung

Schreibe eine Funktion, die Merge Sort implementiert.

****Lösung zu Aufgabe 1:****

1. [8, 4, 7, 3] | [2, 6, 5, 1]
2. [8, 4] [7, 3] | [2, 6] [5, 1]
3. [8] [4] [7] [3] | [2] [6] [5] [1]
4. [4, 8] [3, 7] | [2, 6] [1, 5]
5. [3, 4, 7, 8] | [1, 2, 5, 6]
6. [1, 2, 3, 4, 5, 6, 7, 8] (Sortiert)

****Lösung zu Aufgabe 2 (Python):****

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
```

```
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```